

개요

본 문서는 CSCI103 Lecture 8 강의 자료를 바탕으로 재현 가능한 머신러닝(Reproducible Machine Learning)의 핵심 개념과 실제 적용 방법을 다룹니다. 지난 강의에서 다룬 스케일링, 고가용성, 데이터 아키텍처 등의 복습을 시작으로, 데이터 과학의 과학적 방법론, 머신러닝 모델 개발의 전반적인 흐름, 그리고 AI, 머신러닝, 딥러닝의 차이를 명확히 설명합니다. 특히, ML 파이프라인의 복잡성을 관리하고 효율성을 높이는 데 필수적인 MLflow 프레임워크의 구성 요소(Tracking, Projects, Models, Registry)와 그 활용법을 상세히 다룹니다. 데이터 및 모델 드리프트의 중요성과 대응 방안을 이해하고, 실제 코드 예시를 통해 XGBoost 및 Scikit-learn 모델을 MLflow로 관리하는 실습 과정을 안내합니다. 이 문서는 머신러닝 개발의 전 과정을 이해하고, 특히 재현성과 관리의 중요성을 강조하여 초심자도 ML 프로젝트를 성공적으로 수행할 수 있도록 돕는 것을 목표로 합니다.

핵심 요약

- 지난 강의의 핵심 개념(스케일링, 고가용성, 메달리온 패턴 등)을 복습하고 ML 맥락에서 이해합니다.
- 데이터 과학의 과학적 방법론과 ML 모델 개발의 기본 흐름을 파악합니다.
- AI, 머신러닝, 딥러닝의 관계와 ML 학습 유형을 분류하고 예시를 통해 이해합니다.
- ML 파이프라인 내 데이터 엔지니어, 데이터 과학자, ML 엔지니어의 역할을 명확히 구분합니다.
- ML 개발의 복잡성과 과제를 인식하고, ML 플랫폼의 목표를 이해합니다.
- ML 파이프라인의 핵심 구성 요소(Transformer, Estimator, Evaluator, Metric)를 학습합니다.
- 특징 공학(Feature Engineering)의 중요성과 다양한 특징 생성 기법을 습득합니다.
- MLOps의 개념과 MLflow의 주요 구성 요소(Tracking, Projects, Models, Registry)를 상세히 이해합니다.
- MLflow를 활용하여 모델 실험을 추적하고, 모델을 등록 및 관리하는 방법을 실습합니다.
- 데이터 및 모델 드리프트의 유형과 이를 감지하고 대응하는 전략을 학습합니다.
- 실제 코드 예시를 통해 MLflow를 이용한 XGBoost 및 Scikit-learn 모델 개발 과정을 경험합니다.

용어 정리

핵심 요약

다음 표는 강의에서 다루는 주요 용어들을 초심자도 쉽게 이해할 수 있도록 설명합니다.

용어	쉬운 설명	원어	비고
스케일 아웃	컴퓨터 수를 늘려 성능을 높이는 방식	Scale Out	수평적 확장 (Horizontal Scaling)
스케일 업	컴퓨터 성능(CPU, RAM)을 높여 성능을 높이는 방식	Scale Up	수직적 확장 (Vertical Scaling)
고가용성	시스템이 항상 사용 가능한 상태를 유지하는 능력	High Availability	장애 발생 시에도 서비스 지속
재해 복구	재해 발생 시 시스템을 복구하고 데이터를 복원하는 계획	Disaster Recovery	US East 1 outage 사례
탄력성	워크로드에 따라 자원을 자동으로 늘리거나 줄이는 능력	Elasticity	클라우드의 핵심 장점
브릿지 패턴	다른 데이터 소스에 연결하는 유연한 디자인 패턴	Bridge Pattern	Spark의 커넥터 패턴 예시
메달리온 패턴	데이터 품질에 따라 Bronze, Silver, Gold 계층으로 나누는 아키텍처	Medallion Pattern	멀티홉 패턴 (Multihop Pattern)이라고도 함
BI	과거 데이터를 분석하여 '무슨 일이 일어났는지' 파악	Business Intelligence	의사결정을 위한 요약
BA	미래를 예측하고 '왜 일어났는지' 분석하여 '무슨 일이 일어날지' 예측	Business Analytics	진단 및 예측
레이크하우스	데이터 레이크와 데이터 웨어하우스의 장점을 결합한 아키텍처	Lakehouse	효율적인 데이터 처리 및 ML 지원
과학적 방법론	질문, 가설, 실험, 해석, 공유를 통해 지식을 발전시키는 과정	Scientific Method	데이터 과학의 기반
규칙 기반 시스템	'만약 라면, 하라'와 같은 명시적 규칙으로 작동	Rule-based System	복잡해지면 관리 어려움
AI 기반 시스템	알고리즘과 데이터 학습을 통해 스스로 판단	AI-based System	확률적, 확장성 높음
특징 공학	원본 데이터에서 모델 학습에 유용한 특징을 만드는 과정	Feature Engineering	도메인 지식 중요
AutoML	머신러닝 모델 개발 과정을 자동화하는 기술	Automated ML	빠르지만 제어권이 적음
지도 학습	정답(레이블)이 있는 데이터를 학습하여 예측	Supervised Learning	분류, 회귀
비지도 학습	정답 없이 데이터의 숨겨진 패턴을 찾는 학습	Unsupervised Learning	군집화, 차원 축소
강화 학습	시행착오를 통해 최적의 행동을 학습하는 방식	Reinforcement Learning	자율주행차, 게임 AI
MLOps	머신러닝 모델의 개발부터 배포, 운영, 모니터링까지의 전 과정	Machine Learning Operations	DevOps의 ML 버전
MLflow	머신러닝 수명 주기를 관리하는 오픈소스 플랫폼	MLflow	실험 추적, 모델 관리, 배포
MLflow Tracking	모델 실험의 매개변수, 지표, 아티팩트를 기록하고 비교	MLflow Tracking	재현 가능한 실험의 핵심
MLflow Models	다양한 프레임워크로 만든 모델을 표준 형식으로 저장	MLflow Models	모델의 '맛'(flavor) 지원
MLflow Registry	모델을 중앙에서 관리하고 버전, 스테이지를 제어하는 저장소	MLflow Model Registry	모델의 '진실의 원천'
데이터 드리프트	시간이 지남에 따라 데이터의 통계적 특성이 변하는 현상	Data Drift	모델 성능 저하의 원인
모델 드리프트	모델의 예측 성능이 시간이 지남에 따라 저하되는 현상	Model Drift	재훈련 필요성 시사

핵심 개념 및 원리

지난 강의 복습: ML을 위한 기반 지식

지난 강의에서는 분산 컴퓨팅 환경에서 효율적인 데이터 처리를 위한 여러 개념을 다루었습니다. 이러한 개념들은 머신러닝 모델을 개발하고 운영하는 데 필수적인 기반 지식이 됩니다.

스케일링: 시스템 확장 전략

시스템의 성능을 높이는 방법에는 크게 두 가지가 있습니다.

- **스케일 업 (Scale Up, 수직적 확장):**
 - **한 줄 요약:** 단일 컴퓨터의 성능(CPU, RAM 등)을 높여 처리 능력을 향상시키는 방법입니다.
 - **직관적 예시:** 작은 트럭 한 대가 짐을 많이 싣기 위해 더 큰 엔진을 달거나 적재 공간을 늘리는 것에 비유할 수 있습니다.
 - **기술적 설명:** 기존 서버의 하드웨어 사양을 업그레이드하여 더 많은 작업을 처리할 수 있도록 합니다. 구현이 비교적 간단하지만, 물리적 한계가 존재하며 비용 효율성이 떨어질 수 있습니다.
- **스케일 아웃 (Scale Out, 수평적 확장):**
 - **한 줄 요약:** 여러 대의 컴퓨터를 추가하여 작업을 분산 처리함으로써 전체 시스템의 성능을 높이는 방법입니다.
 - **직관적 예시:** 짐을 더 많이 나르기 위해 트럭 한 대를 더 추가하는 것에 비유할 수 있습니다. 여러 대의 트럭이 동시에 짐을 나르면 더 빠르게 많은 짐을 운반할 수 있습니다.
 - **기술적 설명:** 클러스터에 더 많은 노드(컴퓨터)를 추가하여 컴퓨팅 자원을 늘립니다. 대규모 데이터 처리 및 머신러닝 작업에 필수적이며, 클라우드 환경에서 탄력적으로 자원을 확장할 수 있는 장점이 있습니다.

비기능 요구사항: 시스템의 신뢰성

시스템이 얼마나 잘 작동하는지를 나타내는 중요한 특성들입니다.

- **고가용성 (High Availability):**
 - **한 줄 요약:** 시스템이 장애 발생 시에도 중단 없이 서비스를 제공할 수 있는 능력입니다.
 - **직관적 예시:** 24시간 운영되는 편의점처럼, 언제든지 고객이 방문해도 서비스를 받을 수 있는 상태를 유지하는 것입니다.
 - **기술적 설명:** 시스템의 일부 구성 요소에 문제가 생겨도 전체 서비스가 멈추지 않도록 여러 대의 서버를 병렬로 운영하거나, 자동 장애 조치(failover) 기능을 구현하는 것을 의미합니다. 최근 Amazon US East 1 리전의 대규모 서비스 중단 사태는 고가용성의 중요성을 보여주는 사례입니다.
- **재해 복구 (Disaster Recovery):**
 - **한 줄 요약:** 대규모 재해(예: 자연재해, 대규모 시스템 장애) 발생 시에도 데이터를 보호하고 서비스를 복원하는 계획입니다.
 - **직관적 예시:** 한 도시 전체에 정전이 발생했을 때, 다른 도시의 발전소에서 전력을 공급받아 중요한 시설들이 계속 작동하도록 하는 것과 같습니다.
 - **기술적 설명:** 주 데이터 센터가 완전히 마비되었을 때, 다른 지역의 백업 데이터 센터로 전환하여 서비스를 재개하는 전략입니다. 여러 가용성 영역(Availability Zones)을 활용하여 한 지역의 장애가

전체 서비스에 영향을 미치지 않도록 합니다.

- 탄력성 (Elasticity):
 - 한 줄 요약: 시스템이 워크로드 변화에 따라 컴퓨팅 자원을 자동으로 늘리거나 줄일 수 있는 능력입니다.
 - 직관적 예시: 갑자기 손님이 많아지면 테이블을 더 늘리고 직원을 추가했다가, 손님이 줄면 다시 테이블을 줄이고 직원을 퇴근시키는 식당과 같습니다.
 - 기술적 설명: 클라우드 환경에서 오토스케일링(Autoscaling) 기능을 사용하여, 트래픽이 증가하면 자동으로 서버 인스턴스를 추가하고, 트래픽이 감소하면 인스턴스를 줄여 비용 효율성을 높입니다.

Spark 디자인 패턴: 데이터 처리 효율화

Spark와 같은 분산 처리 프레임워크에서는 특정 문제를 해결하기 위한 디자인 패턴이 존재합니다.

- 브릿지 패턴 (Bridge Pattern) / 커넥터 패턴 (Connector Pattern):
 - 한 줄 요약: 다양한 데이터 소스에 유연하게 연결할 수 있도록 돕는 디자인 패턴입니다.
 - 직관적 예시: 여러 종류의 전자기기(스마트폰, 태블릿, 노트북)를 하나의 범용 충전기(USB-C)로 충전할 수 있는 것과 같습니다.
 - 기술적 설명: Spark에서 JDBC 드라이버 등을 사용하여 관계형 데이터베이스, NoSQL 데이터베이스, 클라우드 스토리지 등 다양한 데이터 소스에 연결할 때 사용됩니다. 데이터 소스 변경 시 코드 수정 없이 유연하게 대응할 수 있습니다.
- 멀티홉 패턴 / 메달리온 패턴 (Multihop Pattern) / (Medallion Pattern):
 - 한 줄 요약: 데이터를 품질과 정제 수준에 따라 여러 계층(Bronze, Silver, Gold)으로 나누어 저장하고 처리하는 아키텍처 패턴입니다.
 - 직관적 예시: 광산에서 캐낸 원석(Bronze)을 정제하여 보석(Silver)으로 만들고, 이를 가공하여 아름다운 장신구(Gold)를 만드는 과정과 같습니다. 각 단계마다 가치가 더해지고 용도가 명확해집니다.
 - 기술적 설명:
 - * **Bronze** (원시 데이터): 원본 데이터를 그대로 저장하는 계층입니다.
 - * **Silver** (정제된 데이터): Bronze 계층의 데이터를 정제, 표준화, 결합하여 ML 모델 학습이나 분석에 적합한 형태로 만듭니다. 데이터 과학자가 주로 활용합니다.
 - * **Gold** (큐레이션된 데이터): Silver 계층의 데이터를 비즈니스 요구사항에 맞춰 집계, 요약하여 BI 대시보드나 최종 사용자 애플리케이션에 제공합니다.
 - 장점:
 - * **모듈성**: 각 계층이 독립적이어서 개발 및 유지보수가 용이합니다.
 - * **견고성**: 한 계층에서 오류가 발생해도 다른 계층에 미치는 영향을 최소화하고, 실패한 계층만 재실행할 수 있습니다.
 - * **다양한 사용자 지원**: 원시 데이터부터 비즈니스 보고서까지 다양한 수준의 데이터를 제공하여 데이터 엔지니어, 데이터 과학자, 비즈니스 분석가 등 여러 사용자의 요구를 충족시킵니다.

BI vs. BA: 데이터 활용의 관점

데이터를 분석하여 비즈니스 의사결정에 활용하는 두 가지 주요 관점입니다.

- 비즈니스 인텔리전스 (BI, Business Intelligence):
 - 한 줄 요약: 과거 데이터를 분석하여 '무슨 일이 일어났는지'를 파악하고 요약하여 의사결정을

드립니다.

- **직관적 예시:** 지난달 매출 보고서를 보고 어떤 제품이 가장 많이 팔렸는지, 어떤 요일에 매출이 높았는지 파악하는 것입니다.
- **기술적 설명:** 대시보드, 보고서 등을 통해 현재 및 과거의 비즈니스 성과를 시각화하고 요약하여, 경영진이 현황을 이해하고 전략을 수립하는 데 필요한 정보를 제공합니다.
- **비즈니스 분석 (BA, Business Analytics):**
 - **한 줄 요약:** 과거 데이터를 바탕으로 '왜 그런 일이 일어났는지'를 진단하고, '미래에 무슨 일이 일어날지'를 예측합니다.
 - **직관적 예시:** 지난달 매출이 감소한 원인을 분석(진단)하고, 다음 분기 매출을 예측(예측)하여 마케팅 전략을 세우는 것입니다.
 - **기술적 설명:** 통계 모델, 머신러닝 알고리즘 등을 활용하여 데이터 간의 관계를 탐색하고, 미래 추세를 예측하며, 최적의 의사결정을 위한 통찰력을 제공합니다.

핵심 요약

BI 도구의 성능을 평가할 때 중요한 두 가지 지표는 다음과 같습니다.

- **속도/지연 시간 (Speed/Latency):** 쿼리 실행 및 결과 도출에 걸리는 시간. 사용자가 빠르게 정보를 얻을 수록 만족도가 높습니다.
- **동시성 (Concurrency):** 동시에 얼마나 많은 사용자를 지원할 수 있는지. 한 명의 분석가뿐만 아니라 수백, 수천 명의 분석가가 동시에 도구를 사용할 수 있어야 합니다.

레이크하우스

(Lakehouse): AI + BI 통합 아키텍처

- **한 줄 요약:** 데이터 레이크(Data Lake)의 유연성과 데이터 웨어하우스(Data Warehouse)의 구조화된 성능을 결합하여, AI와 BI 워크로드를 모두 지원하는 통합 데이터 아키텍처입니다.
- **직관적 예시:** 모든 종류의 물건(정형/비정형 데이터)을 보관할 수 있는 대형 창고(데이터 레이크)에, 물건들을 효율적으로 분류하고 검색할 수 있는 선반 시스템(데이터 웨어하우스 기능)을 추가한 것과 같습니다.
- **기술적 설명:** 단일 플랫폼에서 정형, 반정형, 비정형 데이터를 모두 저장하고, SQL 기반의 효율적인 쿼리, ACID 트랜잭션, 데이터 거버넌스 기능을 제공합니다. 이를 통해 데이터 엔지니어링, BI, 머신러닝 작업을 통합적으로 수행할 수 있습니다.

데이터 과학 및 머신러닝 기초

데이터 과학은 데이터로부터 지식을 추출하고 통찰력을 얻는 학문 분야이며, 머신러닝은 이 과정의 핵심 도구입니다.

데이터 과학의 정의 및 구성 요소

- **한 줄 요약:** 데이터 과학은 과학적 방법론을 사용하여 실제 문제를 해결하고 비즈니스 가치를 창출하는 학제 간 분야입니다.
- **구성 요소:**
 - **응용 통계 및 수학:** 데이터 패턴을 이해하고 모델을 구축하는 데 필요한 이론적 기반.

- **도메인 지식:** 특정 산업(마케팅, 석유 생산 등)에 대한 이해를 바탕으로 문제 정의 및 해결책 도출.
- **컴퓨터 과학 및 소프트웨어 공학:** 데이터 처리, 알고리즘 구현, 시스템 구축에 필요한 기술.
- **목적:** 실제 세계의 문제를 해결하고, 환경에 대한 더 나은 이해를 통해 비즈니스를 개선하며, 궁극적으로 인류의 삶을 향상시키는 것입니다.

과학적 방법론

(Scientific Method) 데이터 과학은 과학적 방법론을 따르며, 이는 지속적인 학습과 개선을 가능하게 합니다.

- **한 줄 요약:** 질문 제기, 가설 설정, 실험, 결과 해석, 공유의 반복을 통해 지식을 확장하는 체계적인 과정입니다.
- **단계:**
 1. **질문 (Question):** "왜 매출이 감소하고 있는가?"와 같이 해결하고자 하는 문제를 명확히 정의합니다.
 2. **가설 (Hypothesis):** 질문에 대한 잠정적인 답변을 제시합니다. "사용자 참여도가 낮아져 매출이 감소하고 있을 것이다."
 3. **실험 (Experimentation):** 가설의 옳고 그름을 검증하기 위한 데이터를 수집하고 분석합니다.
 4. **결과 해석 (Interpretation):** 실험 결과를 분석하여 가설이 맞는지 틀리는지 판단합니다. 가설이 틀렸다면 새로운 가설을 세우고 다시 실험합니다.
 5. **결과 공유 (Sharing Results):** 얻은 지식과 통찰력을 관련 커뮤니티(과학계 또는 회사 내부)와 공유하여 모두가 학습하고 발전할 수 있도록 합니다.

규칙 기반 시스템 vs. AI 기반 시스템

문제를 해결하는 방식에 따라 시스템을 분류할 수 있습니다.

- **규칙 기반 시스템 (Rule-based Systems):**
 - **한 줄 요약:** 명시적으로 정의된 '만약 라면, 하라'와 같은 규칙(if-then-else)과 사실(facts)에 기반하여 작동합니다.
 - **장점:** 특정 문제에 대해 예측 가능하고 명확한 결과를 제공합니다.
 - **단점:** 규칙이 많아지면 관리 및 유지보수가 매우 복잡해지고, 새로운 상황에 유연하게 대처하기 어렵습니다.
- **AI 기반 시스템 (AI-based Systems) / 머신러닝 (Machine Learning):**
 - **한 줄 요약:** 데이터와 알고리즘을 통해 스스로 학습하여 패턴을 발견하고 예측을 수행합니다. 명시적으로 프로그래밍되지 않습니다.
 - **장점:** 대규모 데이터에서 복잡한 패턴을 학습하고, 규칙 기반 시스템보다 확장성이 높으며, 새로운 데이터에 유연하게 적응할 수 있습니다.
 - **단점:** 더 많은 데이터와 컴퓨팅 자원을 필요로 하며, 결과가 확률적이고 해석이 어려울 수 있습니다.

머신러닝 모델 개발 흐름

(ML Model Development Flow) 머신러닝 모델을 개발하는 과정은 다음과 같은 연속적인 단계로 이루어 집니다.

1. 데이터 탐색 (Data Exploration) 및 데이터 정제 (Data Wrangling):
 - 다양한 데이터셋을 수집하고 결합합니다.
 - 누락된 값 채우기, 중복 제거, 필터링 등을 통해 데이터 품질을 개선합니다.
2. 특징 공학 (Feature Engineering):
 - 정제된 데이터에서 모델 학습에 유용한 새로운 특징 (Feature)을 생성합니다.
 - 예: 고객의 평균 거래 금액.
3. 모델 개발 (Model Development):
 - 생성된 특징을 입력으로 사용하여 모델을 학습시킵니다.
 - 모델 평가, 튜닝, 선택 과정을 거쳐 최적의 모델을 찾습니다.
4. 모델 배포 (Model Deployment) 및 추론 (Inferencing):
 - 학습된 모델을 실제 환경에 배포하여 실시간으로 비즈니스 질문에 대한 예측을 수행합니다.
5. 모델 모니터링 및 재훈련 (Model Monitoring & Retraining):
 - 모델 배포 후에도 모델 성능을 지속적으로 모니터링합니다.
 - 모델 드리프트(성능 저하)가 발생하면 새로운 특징을 개발하거나, 더 많은 데이터를 통합하거나, 모델을 재훈련하여 성능을 개선합니다.

예제

ML 파이프라인은 과학적 방법론의 여러 단계에 기여합니다.

- **실험:** 모델을 사용하여 다양한 가설을 테스트할 수 있습니다.
- **결과 해석:** 모델의 예측 결과를 분석하여 가설의 유효성을 판단합니다.
- **결과 공유:** 모델 배포를 통해 예측 결과를 비즈니스에 제공합니다.

최근에는 ChatGPT와 같은 대규모 언어 모델(LLM)이 질문을 생성하고 잠재적인 가설을 제시하는데 도움을 주어 과학적 방법론의 초기 단계에도 기여할 수 있게 되었습니다.

AI vs. 머신러닝 vs. 딥러닝

이 세 가지 개념은 서로 계층적인 관계를 가집니다.

- **AI (인공지능, Artificial Intelligence):**
 - **한 줄 요약:** 인간이 수행하는 작업을 모방하거나 자동화하는 모든 기술을 포괄하는 가장 넓은 개념입니다.
 - **직관적 예시:** 체스 게임을 하는 컴퓨터, 음성 인식 비서, 로봇 청소기 등 인간처럼 생각하거나 행동하는 것처럼 보이는 모든 시스템.
 - **기술적 설명:** 문제 해결, 학습, 의사결정, 패턴 인식 등 인간 지능의 특정 측면을 모방하는 시스템을 만드는 것을 목표로 합니다. 규칙 기반 시스템도 AI의 한 형태입니다.
- **머신러닝 (ML, Machine Learning):**
 - **한 줄 요약:** 데이터를 통해 스스로 학습하여 명시적인 프로그래밍 없이도 작업을 수행하는 AI의 하위 분야입니다.
 - **직관적 예시:** 스팸 메일을 자동으로 분류하는 시스템. 스팸 메일의 특징을 학습하여 새로운 메일이 스팸인지 아닌지 판단합니다.
 - **기술적 설명:** 알고리즘을 사용하여 데이터에서 패턴을 찾아내고, 이를 바탕으로 예측이나 결정을 내립니다. 규칙 기반 시스템과 달리 확률적 접근 방식을 사용하며, 더 많은 데이터가 주어질수록

성능이 향상됩니다.

- **딥러닝 (DL, Deep Learning):**

- **한 줄 요약:** 복잡한 다층 신경망(Neural Networks)을 사용하여 대규모 비정형 데이터에서 패턴을 학습하는 머신러닝의 하위 분야입니다.
- **직관적 예시:** 고양이 사진을 보고 고양이를 인식하거나, 사람의 목소리를 이해하여 대화하는 시스템.
- **기술적 설명:** 인간 뇌의 신경망 구조를 모방한 인공 신경망을 깊게 쌓아 올린 형태로, 이미지, 음성, 텍스트와 같은 비정형 데이터 처리에서 뛰어난 성능을 보입니다. LLM, 자율주행차, Go 게임 챔피언 등 최근 AI의 극적인 성공은 대부분 딥러닝 덕분입니다.

핵심 요약

딥러닝의 급격한 발전은 다음 두 가지 핵심 요소 덕분입니다.

- **대규모 데이터 (Lots of Data):** 딥러닝 모델은 매우 '데이터 굶주림 (data hungry)'이 강하여, 방대한 양의 데이터가 학습에 필수적입니다.
- **고성능 컴퓨팅 (High Compute):** 복잡한 신경망을 학습시키기 위해서는 GPU와 같은 고성능 컴퓨팅 자원이 대량으로 필요합니다. 클라우드 플랫폼(AWS, Databricks 등)의 발전으로 이러한 컴퓨팅 자원이 저렴하고 쉽게 접근 가능해졌습니다.

Spark의 역할: ML 파이프라인의 복잡성 관리

Spark와 같은 분산 처리 플랫폼은 머신러닝 모델 개발 과정의 복잡성을 관리하는 데 중요한 역할을 합니다. 모델 자체를 생성하는 것은 전체 과정의 작은 부분에 불과하며, 데이터 구성, 특징 추출, 전처리 등 많은 단계가 필요합니다. Spark는 이러한 복잡한 데이터 파이프라인을 효율적으로 구축하고 관리할 수 있도록 지원합니다.

머신러닝 프레임워크 및 유형

다양한 머신러닝 문제와 접근 방식이 존재하며, 이를 해결하기 위한 프레임워크와 학습 유형이 있습니다.

ML 프레임워크 종류

- **고전적 및 통계적 알고리즘 (Classical and Statistical Algorithms):** 선형 회귀, 로지스틱 회귀, 의사결정 트리, SVM 등 전통적인 알고리즘.
- **신경망 및 딥러닝 (Neural Net and Deep Learning):** 다층 퍼셉트론, CNN, RNN, 트랜스포머 등 복잡한 신경망 기반 알고리즘.
- **생성형 AI (Generative AI):** LLM(대규모 언어 모델) 및 GAN(생성적 적대 신경망)을 사용하여 새로운 데이터(텍스트, 이미지 등)를 생성.

AutoML vs. 노트북 기반 개발

ML 파이프라인을 구축하는 방식에 따라 다음과 같이 나눌 수 있습니다.

- **AutoML (Automated ML):**
 - **한 줄 요약:** 머신러닝 모델 개발의 여러 단계(데이터 전처리, 특징 선택, 모델 선택, 하이퍼파라미터 튜닝 등)를 자동화하는 기술입니다.

- **장점:** 더 쉽고 빠르며, '시민 데이터 과학자(Citizen Data Scientist)'와 같이 ML 전문 지식이 부족한 사람도 모델을 개발할 수 있도록 돕습니다.
- **단점:** 제어권이 적고 사용자 정의(customization)가 제한적입니다.
- **노트북 기반 개발 (Notebook-based Development):**
 - **한 줄 요약:** Jupyter Notebook과 같은 환경에서 코드를 직접 작성하여 모델을 개발하는 방식입니다.
 - **장점:** 더 강력하고 유연하며, 모델 개발의 모든 측면을 세밀하게 제어할 수 있습니다. **단점:** 더 복잡하고 높은 수준의 기술 역량을 요구합니다.

Databricks의 'Glass Open AutoML'은 AutoML의 장점(쉬움)과 노트북 기반 개발의 장점(제어권)을 결합하여, AutoML이 생성한 코드를 사용자가 직접 수정하고 개선할 수 있도록 합니다.

ML 학습 유형 분류

머신러닝 문제는 데이터의 특성과 해결하려는 문제의 종류에 따라 다양하게 분류될 수 있습니다.

- **이산 (Discrete) vs. 연속 (Continuous):**
 - **이산:** 예측 결과가 정해진 범주(예: 비가 올까/안 올까, 스팸/정상) 중 하나인 경우.
 - **연속:** 예측 결과가 연속적인 숫자 값(예: 내일 강수량, 집값)인 경우.
- **지도 학습 (Supervised Learning) vs. 비지도 학습 (Unsupervised Learning):**
 - **지도 학습:** 정답(레이블)이 있는 데이터를 학습하여 새로운 데이터에 대한 예측을 수행합니다.
 - **비지도 학습:** 정답 없이 데이터 자체의 숨겨진 패턴이나 구조를 찾아냅니다.

예제

- **내일 비가 얼마나 올까?**
 - **유형:** 지도 학습 (Supervised Learning) + 연속 (Continuous)
 - **설명:** 과거의 날씨 데이터(기온, 습도 등)와 실제 강수량(정답)을 학습하여 내일의 강수량이라는 연속적인 값을 예측합니다. 이는 **회귀(Regression)** 문제에 해당합니다.
- **내일 비가 올까?**
 - **유형:** 지도 학습 (Supervised Learning) + 이산 (Discrete)
 - **설명:** 과거 날씨 데이터와 비가 왔는지/안 왔는지(정답)를 학습하여 내일 비가 올지 안 올지(예/아니오)를 예측합니다. 이는 **분류(Classification)** 문제에 해당합니다.
- **날씨 패턴에 따라 요일을 그룹화할 수 있을까?**
 - **유형:** 비지도 학습 (Unsupervised Learning) + 이산 (Discrete)
 - **설명:** 요일별 날씨 데이터에서 유사한 패턴을 가진 요일들을 그룹으로 묶습니다(예: 맑은 요일 그룹, 흐린 요일 그룹). 이는 **군집화(Clustering)** 문제에 해당합니다.
- **소비자 구매에 영향을 미치는 핵심 요인은 무엇일까?**
 - **유형:** 비지도 학습 (Unsupervised Learning) + 연속 (Continuous) 또는 이산 (Discrete)
 - **설명:** 수많은 잠재적 요인(가격, 가용성, 브랜드 인지도, 날씨 등) 중에서 실제 구매에 가장 큰 영향을 미치는 몇 가지 핵심 요인을 찾아냅니다. 이는 **차원 축소(Dimensionality Reduction)** 문제에 해당합니다.

핵심 요약

- **회귀 (Regression)**: 연속적인 숫자 값을 예측합니다 (예: 주택 가격 예측).
- **분류 (Classification)**: 데이터를 미리 정의된 범주 중 하나로 분류합니다 (예: 의료 영상에서 암 진단, 스팸 메일 분류).
- **군집화 (Clustering)**: 유사한 특성을 가진 데이터 포인트를 그룹으로 묶습니다 (예: 고객 세분화, 고속도로 차선 감지).
- **연관 규칙 (Association)**: 데이터셋에서 항목 간의 흥미로운 관계를 찾습니다 (예: 기저귀와 맥주 구매 연관성).
- **강화 학습 (Reinforcement Learning)**: 에이전트가 환경과 상호작용하며 시행착오를 통해 최적의 행동 정책을 학습합니다. 명확한 정답이 아닌, 보상 시스템을 통해 장기적인 목표를 달성합니다 (예: 자율주행차, 게임 AI).

주의

더 많은 데이터가 더 복잡한 알고리즘보다 낫다. 일반적으로 모델의 예측 성능을 향상시키는 데 있어, 더 많은 양의 데이터를 확보하는 것이 복잡한 알고리즘을 미세 조정하는 것보다 더 큰 영향을 미칩니다. 데이터는 종종 알고리즘 튜닝보다 훨씬 더 큰 결과를 가져옵니다.

ML 역할 및 플랫폼

머신러닝 모델을 성공적으로 개발하고 운영하기 위해서는 다양한 전문가의 협업과 효율적인 플랫폼이 필요합니다.

ML 파이프라인 내 역할

ML 파이프라인은 여러 전문가의 협업을 통해 구축됩니다.

- **데이터 엔지니어 (Data Engineer)**:
 - **역할**: 데이터 수집 (Ingesting), 저장 (Storing), 검증 (Validating), 정제 (Cleaning), 표준화 (Standardizing)를 담당합니다.
 - **산출물**: 정제된 큐레이션 데이터 (메달리온 아키텍처의 Silver 계층)를 데이터 과학자에게 전달합니다.
- **데이터 과학자 (Data Scientist)**:
 - **역할**: 데이터 엔지니어가 제공한 데이터를 바탕으로 특징 (Feature)을 계산하고 선택합니다. 모델을 코딩하고 훈련하며, 검증 작업을 수행합니다.
 - **산출물**: 학습된 모델과 모델 평가 결과.
- **ML 엔지니어 (ML Engineer)**:
 - **역할**: 데이터 과학자가 개발한 모델을 검증하고 평가합니다. 모델을 패키징 (Packaging), 컨테이너화 (Containerizing), 배포 (Deploying)하여 실시간 추론 (Inferencing)이 가능하도록 서비스합니다.
 - **산출물**: 배포된 모델 서비스.

이러한 역할들은 연속적인 사이클을 형성하며, 모델이 배포된 후에도 지속적으로 데이터를 수집하고 모델을 개선하는 과정이 반복됩니다.

ML 플랫폼의 목표

(Aspirations of ML Platform) 이상적인 ML 플랫폼은 다음과 같은 목표를 달성해야 합니다.

- **다양한 도구 지원:** 다양한 라이브러리, 언어, 협업 방식을 지원해야 합니다.
- **다양한 배포 전략 지원:** 다양한 유형의 컴퓨팅 자원(단일 노드, 분산 알고리즘, CPU, GPU)을 지원해야 합니다.
- **높은 효율성:** 훈련, 튜닝, 추적, 추론 과정에서 낮은 비용과 빠른 속도를 제공해야 합니다.
- **특징 저장소 (Feature Store):** 데이터 과학자가 생성한 특징 (Feature)을 관리하고 쉽게 재사용할 수 있도록 저장해야 합니다.
- **AutoML 지원:** 시민 데이터 과학자를 위해 ML 파이프라인 생성을 간소화해야 합니다.
- **모델 드리프트 모니터링:** 모델에 입력되는 데이터가 훈련 데이터와 달라지거나 모델 성능이 저하되는 현상(드리프트)을 감지하고 재훈련 필요성을 알려야 합니다.
- **ML 파이프라인 운영화 (Operationalization):** ML 파이프라인을 배포하고 사용자들이 쉽게 접근하여 추론할 수 있도록 해야 합니다.
- **재현성 (Reproducibility):** 동일한 데이터와 코드를 사용하여 모델을 다시 생성했을 때 동일한 결과를 얻을 수 있도록 보장해야 합니다.

ML 수명 주기 및 과제

ML 모델은 한 번 만들고 끝나는 것이 아니라, 지속적인 관리와 개선이 필요한 '살아있는 자산'입니다.

ML 수명 주기 단계

ML 모델의 수명 주기는 다음과 같은 핵심 단계로 구성됩니다.

1. **데이터 수집 및 정제 (Data Ingestion & Cleansing):** 원시 데이터를 모델 학습에 적합한 형태로 만듭니다.
2. **특징화 (Featurization):** 정제된 데이터에서 모델에 필요한 특징을 추출하거나 생성합니다.
3. **모델 훈련 (Model Training):** 특징 데이터를 사용하여 모델을 학습시킵니다.
4. **모델 서빙 (Model Serving) / 추론 (Inference):** 학습된 모델을 배포하여 새로운 데이터에 대한 예측을 수행합니다. 배치 (Batch), 실시간 (Real-time), HTTP 요청 등 다양한 방식으로 이루어질 수 있습니다.
5. **모델 모니터링 (Model Monitoring):** 배포된 모델의 성능을 지속적으로 감시하고 드리프트(성능 저하)를 감지합니다.

이러한 단계들은 선형적으로 진행되기도 하지만, 모니터링 결과에 따라 이전 단계(데이터 정제, 특징화, 모델 훈련)로 돌아가 개선 작업을 수행하는 반복적인 사이클을 이룹니다.

ML 개발의 복잡성 및 과제

ML 프로젝트는 다음과 같은 이유로 빠르게 복잡해지고 전문화된 지식을 요구합니다.

- **파편화된 생태계 (Fragmented Ecosystem):** Python, Spark, R 등 다양한 언어와 수많은 라이브러리 (TensorFlow, PyTorch, Scikit-learn, XGBoost 등)가 존재하여, 특정 프레임워크에 대한 숙련도가 생산성에 큰 영향을 미칩니다.
- **모델 튜닝의 어려움 (Model Tuning Complexity):** 모델 학습뿐만 아니라 하이퍼파라미터 최적화

(Hyperparameter Optimization)는 여러 번의 반복과 실험을 필요로 하는 비선형적인 과정입니다.

- **배포의 비자명성 (Non-trivial Deployment):** 모델을 개발하는 것과 실제 운영 환경에 배포하여 안정적으로 서비스하는 것은 별개의 문제입니다. 많은 ML 프로젝트가 프로덕션 단계로 넘어가지 못하는 이유입니다.
- **모델 관리의 필요성 (Model Management):** 모델은 '살아있는 자산'이므로, 한 번 배포했다고 끝나는 것이 아니라 지속적인 관리, 개선, 정당화가 필요합니다. 모델의 예측이 비즈니스 의사결정에 직접적인 영향을 미치기 때문입니다.
- **협업의 중요성 (Collaboration):** 데이터 엔지니어, 데이터 과학자, ML 엔지니어 등 다양한 역할 간의 원활한 협업(Hand-on, Hand-off)이 필수적입니다.
- **확장성 (Scale):** 노트북에서 개발한 모델이 대규모 프로덕션 데이터 볼륨을 처리할 수 있도록 확장되어야 합니다.
- **거버넌스 (Governance):** 모델이 민감한 데이터를 사용하여 훈련될 수 있으므로, 데이터 거버넌스(Unity Catalog)와 유사하게 모델에 대한 접근 제어 및 감사(Audit)가 필요합니다. 모델의 공정성(Fairness)과 설명 가능성(Explainability)은 규제 준수 측면에서 중요합니다.

핵심 요약

- **특징 저장소 (Feature Store):** 특징을 저장하고 재사용하기 위한 중앙 저장소가 필요합니다.
- **실험 추적 (Experiment Tracking):** 수많은 실험의 매개변수, 지표, 결과를 자동으로 기록하고 재현 가능하도록 관리해야 합니다.
- **AutoML:** 데이터 과학자의 작업을 간소화하지만, 제어권이 부족할 수 있습니다. Databricks의 'Glass Open AutoML'은 생성된 코드를 수정할 수 있어 이러한 단점을 보완합니다.
- **클라우드 실행 (Cloud Execution):** 클라우드 환경에서 효율적으로 ML 워크로드를 실행해야 합니다.
- **AB 테스트 (AB Testing):** 새로운 모델이 기존 모델(챔피언)보다 우수한지 검증하기 위해 점진적으로 트래픽을 전환하는 과정이 필요합니다.
- **CI/CD (Continuous Integration/Continuous Deployment):** ML 모델 개발도 소프트웨어 개발처럼 지속적인 통합 및 배포 파이프라인에 통합되어야 합니다.
- **오케스트레이션 (Orchestration):** 데이터 파이프라인, ML 모델 훈련, 추론, BI 대시보드 등 여러 구성 요소를 하나의 워크플로우로 관리해야 합니다.
- **데이터/모델 드리프트 (Data/Model Drift):** 데이터 특성이나 모델 성능 변화를 지속적으로 모니터링하고 대응해야 합니다.

ML 파이프라인 구성 요소

ML 파이프라인은 데이터 처리 및 모델 학습을 위한 일련의 연결된 단계로 구성됩니다.

- **트랜스포머 (Transformer):**
 - **한 줄 요약:** 입력 데이터를 받아 변환하여 다른 형태의 데이터를 출력하는 구성 요소입니다. 학습 과정이 없습니다.
 - **직관적 예시:** 원본 사진(입력)을 흑백 사진(출력)으로 바꾸는 필터와 같습니다. 필터 자체는 학습하지 않습니다.
 - **기술적 설명:** 데이터 정제, 특징 추출, 스케일링 등 데이터 전처리 작업에 사용됩니다. 입력은 데이

터프레임이고 출력도 데이터프레임입니다.

– 주요 예시:

- * **StringIndexer**: 문자열 범주형 데이터를 숫자 인덱스로 변환.
- * **OneHotEncoder**: 숫자 인덱스를 원-핫 인코딩 벡터로 변환 (범주형 변수 처리).
- * **StandardScaler**: 데이터를 표준 정규 분포 (평균 0, 표준편차 1)로 스케일링하여 정규화.
- * **Imputer**: 데이터셋의 결측값을 채웁니다 (평균, 중앙값, 최빈값 등).
- * **VectorAssembler**: 여러 특징 컬럼을 하나의 벡터 컬럼으로 결합하여 모델 입력 형식에 맞춥니다.

• **에스티메이터 (Estimator)**:

- **한 줄 요약**: 입력 데이터를 학습하여 모델을 생성하는 구성 요소입니다. 'fit' 함수를 호출할 때 학습이 일어납니다.
- **직관적 예시**: 학생(에스티메이터)이 교과서(입력 데이터)를 공부하여 시험 문제(새로운 데이터)를 풀 수 있는 지식(모델)을 얻는 것과 같습니다. **기술적 설명**: 머신러닝 알고리즘(예: 선형 회귀, 의사결정 트리) 자체를 의미합니다. 입력은 데이터프레임이고 출력은 학습된 모델입니다.

• **평가자 (Evaluator) 및 지표 (Metric)**:

- **한 줄 요약**: 모델의 성능을 측정하고 개선 방향을 제시하는 도구와 기준입니다.
- **직관적 예시**: 시험 점수(지표)를 통해 학생의 학습 수준(모델 성능)을 평가하고, 어떤 과목을 더 공부해야 할지(개선 방향) 알려주는 것과 같습니다.
- **기술적 설명**: 모델의 예측 결과와 실제 정답을 비교하여 정확도(Accuracy), F1-Score, RMSE(Root Mean Squared Error) 등 다양한 지표를 계산합니다. 이러한 지표는 모델의 개선 여부를 판단하는데 필수적입니다.
- **주요 지표 예시**:
 - * **정확도 (Accuracy)**: 전체 예측 중 올바르게 예측한 비율.
 - * **F1-Score**: 정밀도(Precision)와 재현율(Recall)의 조화 평균. 불균형 데이터셋에서 유용합니다.
 - * **혼동 행렬 (Confusion Matrix)**: 분류 모델의 성능을 시각적으로 보여주는 표입니다.
 - **True Positive (TP)**: 실제 양성을 양성으로 올바르게 예측.
 - **True Negative (TN)**: 실제 음성을 음성으로 올바르게 예측.
 - **False Positive (FP)**: 실제 음성을 양성으로 잘못 예측 (1종 오류).
 - **False Negative (FN)**: 실제 양성을 음성으로 잘못 예측 (2종 오류).
 혼동 행렬의 모든 값을 이해하는 것은 특정 도메인(예: 의료 진단)에서 오탐(FP)이나 미탐(FN)의 비용이 매우 클 수 있기 때문에 중요합니다.

특징 공학

(Feature Engineering) 특징 공학은 머신러닝 모델의 성능을 결정하는 가장 중요한 요소 중 하나입니다.

정의 및 중요성

- **한 줄 요약**: 원본 데이터(raw data)에서 머신러닝 모델이 학습하기에 더 유용하고 의미 있는 특징(Feature)을 생성하는 과정입니다.
- **직관적 예시**: 요리사가 신선한 재료(원본 데이터)를 가지고 요리(모델 학습)하기 전에, 재료를 손질

하고(정제), 양념을 만들고(특징 생성), 적절한 크기로 자르는(변환) 과정과 같습니다. 좋은 양념과 손질은 요리의 맛(모델 성능)을 결정합니다.

- **기술적 설명:** 데이터 엔지니어링과 ML의 교차점에 있으며, 도메인 지식과 창의성이 요구됩니다. 단순히 테이블의 컬럼을 사용하는 것을 넘어, 여러 컬럼을 조합하거나 변환하여 모델의 예측력을 높이는 새로운 변수를 만듭니다.
- **중요성:**
 - **모델 성능 향상:** 모델이 데이터의 숨겨진 패턴을 더 잘 이해하도록 돕습니다.
 - **계산 비용 절감:** 특징 생성은 계산 집약적인 작업이므로, 한 번 생성된 특징은 훈련 시간과 추론 시간 모두에서 재사용되어야 합니다.
 - **일관성 유지:** 훈련 시 사용된 특징과 추론 시 사용된 특징이 동일한 방식으로 계산되어야 모델 드리프트를 방지할 수 있습니다.
 - **재사용성:** 사용자 프로필과 같은 공통 특징은 여러 ML 사용 사례에서 재사용될 수 있습니다.

특징 생성 유형 및 예시

- **변환 (Transformations):** 기존 컬럼을 직접 변환합니다.
 - **예시:** 우편번호(zip code)를 범주형 인코딩(category encoding)으로 변환.
- **컨텍스트 특징 (Context Features):** 기존 데이터에 외부 정보를 추가합니다.
 - **예시:** 구매 데이터에 요일(weekday) 정보를 추가하여 특정 요일의 구매 패턴을 분석.
- **특징 증강 (Feature Augmentation):** 외부 데이터 소스를 통합하여 특징을 풍부하게 만듭니다.
 - **예시:** 판매 데이터에 해당 날짜의 날씨 정보를 추가하여 날씨가 판매에 미치는 영향을 분석.
- **사전 계산 특징 (Precomputed Features):** 계산 비용이 많이 드는 특징을 미리 계산하여 저장합니다.
 - **예시:** 고객 생애 가치(Customer Lifetime Value), 지난 7일/14일/21일/월/년 단위의 구매 이력(rolling values). 이러한 특징은 고객의 다음 구매 시점을 예측하는 데 유용하며, 계산 비용이 매우 높습니다.

핵심 요약

특징 저장소 (Feature Store)는 데이터 과학자가 생성한 특징을 중앙에서 관리하고 공유하며 재사용할 수 있도록 돕는 시스템입니다. 이를 통해 특징 생성의 일관성을 유지하고, 중복 계산을 방지하며, 특징의 발견 가능성을 높일 수 있습니다.

MLOps

(Machine Learning Operations) MLOps는 머신러닝 모델의 개발부터 배포, 운영, 모니터링까지의 전 과정을 자동화하고 관리하는 방법론입니다.

DataOps, MLOps, ModelOps 정의

- **DevOps:** 소프트웨어 개발(Development)과 운영(Operations)을 통합하여 소프트웨어 배포를 자동화하고 효율화하는 방법론입니다.
- **DataOps:** 데이터 파이프라인의 설계, 구축, 운영을 자동화하고 관리하여 데이터의 품질과 가용성을 높이는 방법론입니다. 데이터 정제, 큐레이션, 골드 테이블 생성 등을 포함합니다.
- **MLOps:** 머신러닝 모델의 개발, 훈련, 배포, 모니터링, 재훈련 등 전체 수명 주기를 자동화하고 관리

하는 방법론입니다. DataOps와 DevOps의 원칙을 ML에 적용한 것입니다.

- **ModelOps:** MLOps의 하위 개념으로, 특히 훈련된 모델이 실제 데이터에 대해 추론(scoring)을 수행하고, 모델 교체 및 버전 관리를 포함하는 운영 단계에 초점을 맞춥니다.

MLOps는 데이터 엔지니어의 데이터 파이프라인(DataOps)과 데이터 과학자의 모델 개발 코드(DevOps)를 통합하고, 모델의 검증, 배포, 추적, 관리를 포함하는 포괄적인 접근 방식입니다.

MLflow 개요

MLflow는 머신러닝 수명 주기를 관리하기 위한 오픈소스 플랫폼입니다.

MLflow란 무엇인가?

- **한 줄 요약:** Databricks에서 개발한 오픈소스 프레임워크로, 머신러닝 실험 추적, 코드 패키징, 모델 배포 및 관리를 위한 표준화된 도구를 제공합니다.
- **인기 요인:** MLflow는 매우 인기가 많아 Databricks 플랫폼뿐만 아니라 경쟁사에서도 표준으로 채택될 정도로 강력한 기능을 제공합니다.
- **오픈소스:** 누구나 자유롭게 사용할 수 있으며, Databricks 플랫폼에서 사용하면 Unity Catalog와의 통합, 관리형 추적 서버 등 추가적인 이점을 얻을 수 있습니다.
- **특징:**
 - **프레임워크 및 도구 불가지론적 (Framework and Tool Agnostic):** TensorFlow, PyTorch, Scikit-learn 등 어떤 ML 프레임워크나 알고리즘을 사용하든 상관없이 작동합니다.
 - **API 우선 (API First):** REST API를 기반으로 구축되어 어떤 라이브러리나 언어에서도 실행(run)이나 모델을 제출할 수 있습니다.
 - **모듈형 (Modular):** 여러 구성 요소(Tracking, Projects, Models, Registry)로 나뉘어 있어, 필요에 따라 개별적으로 사용하거나 전체를 통합하여 사용할 수 있습니다.
 - **쉬운 시작 및 확장성:** 로컬 환경에서 단일 개발자가 쉽게 시작할 수 있으며, 대규모 팀에서도 효과적으로 사용할 수 있도록 확장성이 뛰어납니다.
 - **생태계 우선 (Ecosystem First):** 언어 런타임에 대한 편향이 없어 다양한 프레임워크에서 쉽게 실행됩니다.

MLflow의 주요 구성 요소

MLflow는 다음과 같은 네 가지 주요 구성 요소로 이루어져 있습니다.

1. MLflow Tracking (실험 추적):

- **한 줄 요약:** 머신러닝 실험의 매개변수(parameters), 지표(metrics), 아티팩트(artifacts), 메타데이터(metadata)를 기록하고 시각화하여 실험 과정을 추적합니다.
- **목적:** 데이터 과학자가 수많은 실험을 수행하면서 어떤 매개변수로 어떤 결과를 얻었는지 잊어버리지 않도록 돕고, 재현 가능한 실험을 가능하게 합니다.

2. MLflow Projects (코드 패키징):

- **한 줄 요약:** 머신러닝 코드를 재현 가능한 형식으로 패키징하여 어떤 컴퓨팅 플랫폼에서도 실행할 수 있도록 합니다.
- **목적:** 다른 환경에서 코드를 실행할 때 필요한 라이브러리 종속성(requirements.txt) 및 환경 설정을

자동으로 관리하여, '내 컴퓨터에서는 되는데...'와 같은 문제를 방지합니다.

3. MLflow Models (모델 표준화):

MLflow 상세

MLflow의 각 구성 요소는 ML 수명 주기의 특정 단계를 지원합니다.

MLflow Tracking: 실험의 기록자

MLflow Tracking은 모든 실험의 세부 사항을 기록하여 재현성을 보장합니다.

- **실험 (Experiment):** 특정 가설을 검증하기 위한 일련의 시도.
 - **실행 (Run):** 각 실험 내에서 수행되는 개별적인 모델 훈련 시도. 각 실행은 고유한 ID를 가집니다.
- 기록되는 정보:**
- **매개변수 (Parameters):** 모델 훈련에 사용된 입력 값 (예: 학습률, 최대 깊이).
 - **지표 (Metrics):** 모델의 성능을 나타내는 수치 (예: 정확도, RMSE, F1-Score).
 - **아티팩트 (Artifacts):** 모델 파일 자체, 학습 데이터, 시각화 그래프(PNG), 로그 파일 등 실험과 관련된 모든 출력물.
 - **메타데이터 (Metadata):** 실행 시간, 실행자, 코드 소스, 사용된 데이터셋 등 실험에 대한 추가 정보.
 - **AutoLog:** MLflow는 `autolog()` 기능을 제공하여, 주요 ML 라이브러리(Scikit-learn, TensorFlow 등)의 매개변수와 지표를 자동으로 기록할 수 있습니다. 추가적인 정보는 `mlflow.log_param()`,
 - **UI (사용자 인터페이스):** MLflow Tracking UI를 통해 모든 실험과 실행을 시각적으로 탐색하고 비교할 수 있습니다. 각 실행의 세부 정보(매개변수, 지표, 아티팩트, 메타데이터)를 확인할 수 있으며, 코드가 어디서 왔는지 추적할 수 있어 재현성을 높입니다.

MLflow Models: 모델의 표준화된 표현

MLflow Models는 다양한 ML 프레임워크로 생성된 모델을 표준화된 형식으로 저장하여, 배포 및 사용을 용이하게 합니다.

MLflow Model Registry: 모델의 중앙 관리소

MLflow Model Registry는 조직 내 모델을 중앙에서 관리하고 협업을 지원하는 핵심 구성 요소입니다.

모델 및 데이터 드리프트

ML 모델은 시간이 지남에 따라 성능이 저하될 수 있으며, 이를 '드리프트(Drift)'라고 합니다. 드리프트는 모델 운영에서 가장 중요한 모니터링 대상입니다.

드리프트의 원인 및 중요성

드리프트 유형

드리프트는 여러 가지 형태로 나타날 수 있으며, 원인에 따라 대응 방식이 달라집니다.

핵심 요약

MLflow는 일반적으로 메달리온 아키텍처의 **Silver 계층**에 통합됩니다. 데이터 엔지니어가 Bronze 계층의 원시 데이터를 정제하여 Silver 계층에 저장하면, 데이터 과학자는 이 Silver 계층의 데이터를 MLflow를 사용하여 모델 훈련에 활용합니다. 경우에 따라 Gold 계층의 데이터를 사용할 수도 있습니다.